

Technical Architecture & Developer Handbook

System design, source map, data flows, security, testing and deployment

Document	DocuLens project reference
Edition	1.0 — generated from supplied project archive
Repository	https://github.com/Devputta/DocuLens
Scope	Documented implementation and practical operating guidance

Important: This handbook describes the supplied source archive and its documentation. It does not claim that every feature has been independently audited or that the hosted deployment is currently healthy.

Document control and scope

This handbook is the engineering-facing reference for DocuLens, a local-first document intelligence workbench. It consolidates architecture, source layout, data flow, security boundaries, build and deployment practices, and maintenance guidance.

- Use it alongside the repository README and docs/ directory.
- Treat implementation details as version-specific; confirm the source before changing behavior.
- The document intentionally distinguishes implemented design from operational assumptions.

Artifact	Purpose
README.md	Project overview, capabilities and quick start
docs/ARCHITECTURE.md	Trust boundaries and system design
docs/DEPLOYMENT.md	Hosting and environment configuration
docs/TESTING.md	Verification approach
SECURITY.md	Security reporting and handling

Product and engineering goals

DocuLens is designed to help a user inspect and reason over documents while preserving local control over source files. Its differentiators are provenance, inspectable citations, offline fallback, and exports that retain context.

- Local-first processing reduces unnecessary document transfer.
- Citations are intended to connect answers to document evidence.
- The interface supports reading, search, inquiry, extraction, comparison and backup.
- Remote synthesis is optional and should be treated as an explicit privacy boundary.

System architecture at a glance

The browser is the primary processing and persistence environment. The Express server provides optional remote AI proxy endpoints and citation guardrails; it is not intended to be a document repository.

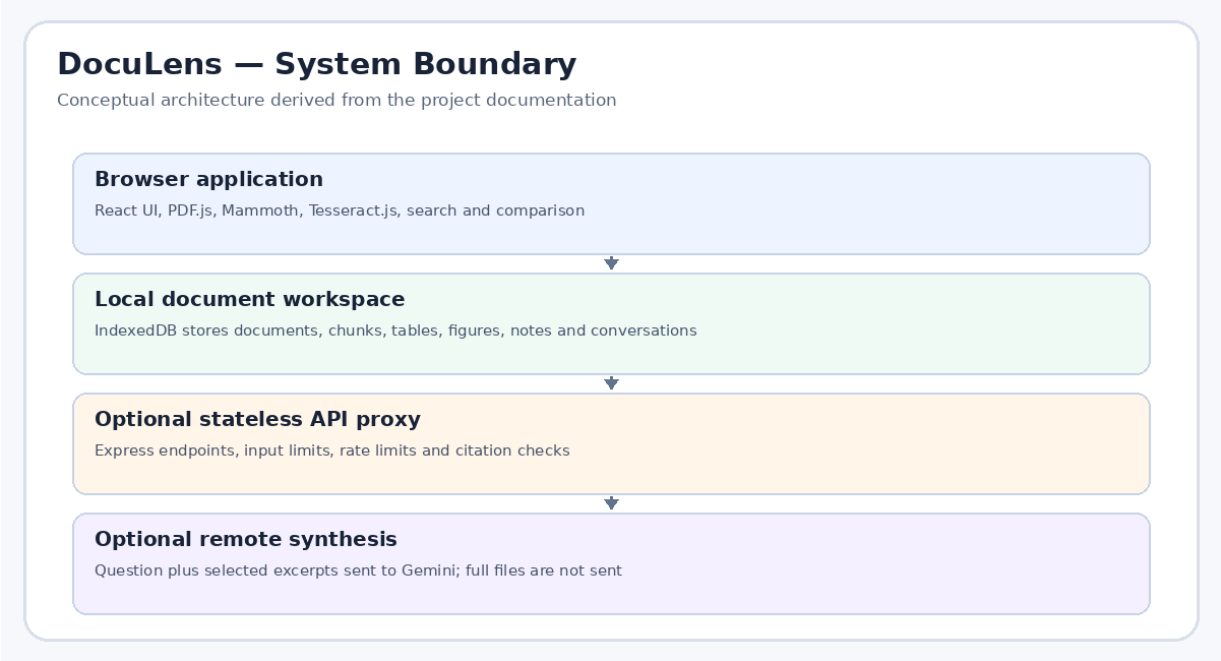


Figure 1. Conceptual client/server architecture.

Technology stack and responsibility map

The repository combines a React interface with Vite tooling and an Express/TypeScript server. Parsing and OCR libraries run in the browser for local document handling.

- React and React DOM: interface rendering.
- Vite and the React plugin: development/build pipeline.
- Express and compression: server routes and response compression.
- PDF.js: PDF parsing and page-level text access.
- Mammoth: DOCX content extraction.
- Tesseract.js: local OCR for scanned pages or figures.
- IndexedDB-backed services: browser-local persistence.

Area	Packages / modules	Primary role
UI	React, Motion, lucide-react	Views, navigation and interaction
Document parsing	pdfjs-dist, mammoth	PDF and DOCX extraction
OCR	tesseract.js, eng.traineddata	Recognize text in image-based content
Server	express, compression, dotenv	Proxy endpoints and configuration
Testing	vitest, fake-indexeddb	Automated checks and storage simulation

Repository map

Use the source tree as the first reference when debugging. Keep UI components, service logic, domain types, and tests separated so changes remain traceable.

- src/components/ contains feature-level interface components.
- src/services/ contains parsing, search, storage, comparison, export and inquiry logic.
- src/types/ holds shared domain types.
- src/utils/ contains small reusable helpers.
- tests/ contains regression tests for document handling and related features.
- docs/ contains architecture, deployment, testing, user, privacy and troubleshooting notes.

Path	What to inspect
server.ts	HTTP middleware, routes, proxy and citation validation
src/App.tsx	Top-level app composition and state wiring
src/services/storage.ts	Persistence operations
src/services/ocrService.ts	OCR setup and language model loading
src/services/lexicalSearch.ts	Search ranking
src/services/comparisonEngine.ts	Comparison algorithms
tests/	Expected behavior and regression coverage

Runtime modes and entry points

The package scripts distinguish development, production build, and production serving. Verify the deployed commands against package.json rather than copying an old dashboard setting.

- npm run dev starts the TypeScript server in development mode.
- npm run build runs Vite after copying OCR assets.
- npm start / bun run start starts server.ts with NODE_ENV=production.
- npm run verify chains lint, tests and build.
- The package declares Node.js >=20.

Script	Meaning
dev	tsx server.ts
start	NODE_ENV=production tsx server.ts
prebuild	node scripts/copy-ocr-assets.mjs
build	vite build
preview	vite preview
verify	npm run lint && npm test && npm run build
clean	rm -rf dist server.js
lint	tsc --noEmit
test	vitest run

Document ingestion pipeline

Ingestion validates a selected file, extracts text or OCR, organizes the result into page/element/chunk records, and saves usable content in local storage. Badly scanned or unusual layouts may need manual review.

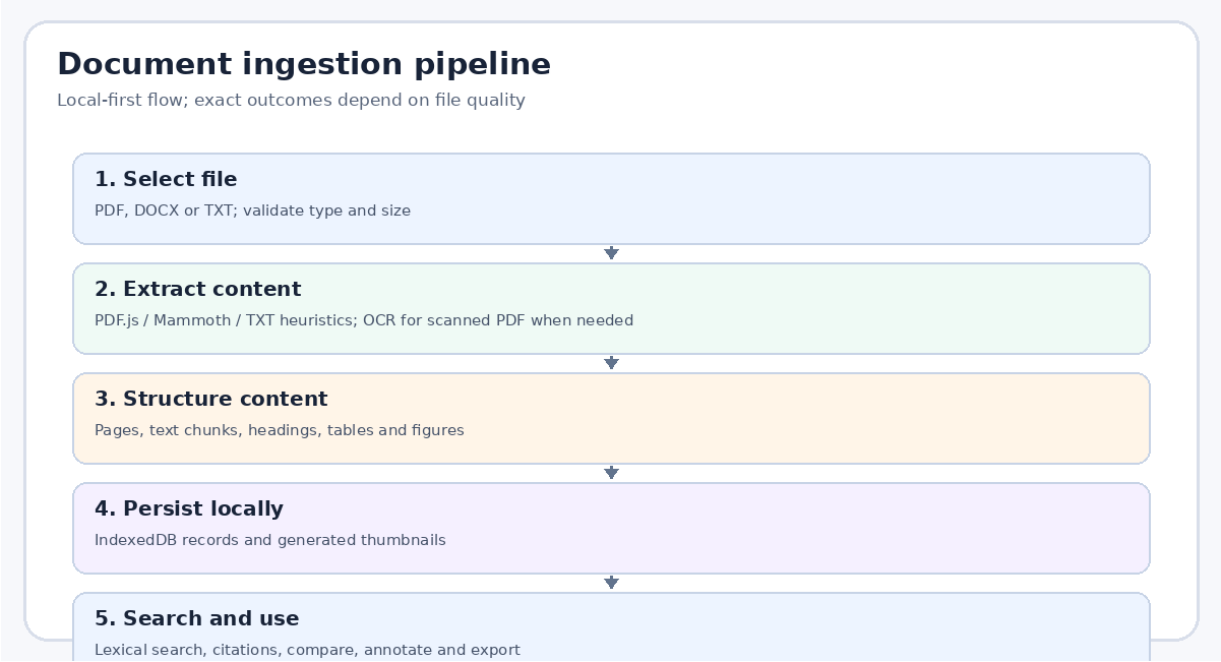


Figure 2. Conceptual ingestion stages.

Supported file types and boundaries

The project documentation describes PDF, DOCX and TXT ingestion. Limits and parsing behavior should be surfaced to users instead of failing silently.

- PDF: documented maximum 25 MB and 500 pages; validates a PDF signature.
- DOCX: parsed locally using Mammoth; encrypted compound binary inputs are blocked.
- TXT: paragraph chunking and heading heuristics; page numbers are virtual/reflowed.
- Scanned PDFs can invoke local Tesseract OCR using the bundled English language data.

Type	Processing path	Caveat
PDF	PDF.js, page extraction and optional OCR	Scans may have poor OCR quality
DOCX	Mammoth conversion	Complex layout may not survive text extraction
TXT	Paragraph and heading heuristics	Pagination is virtual, not original page layout

Browser storage and data model

IndexedDB is the local authority for document workspace data. The project describes records for documents, chunks, elements, tables, figures, notes, conversations and inquiries.

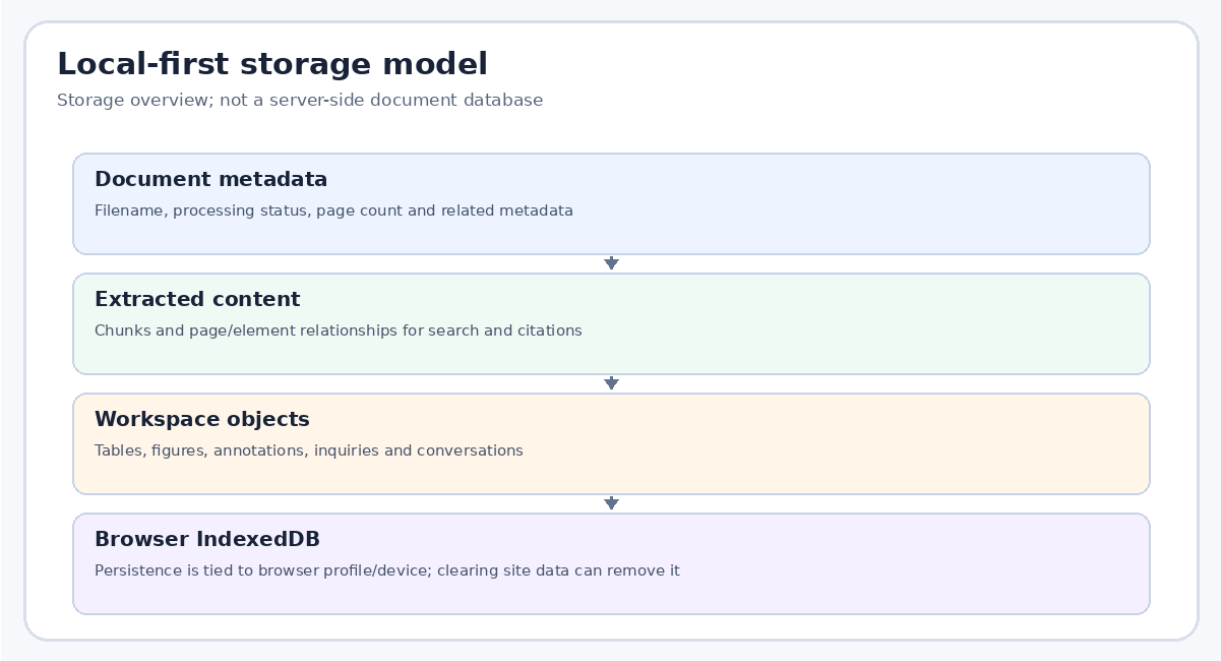


Figure 3. Local-first storage overview.

- Local storage is scoped to the browser profile and origin.
- Private/incognito sessions and clearing site data may remove local records.
- Back up important work before browser resets, device migration or reinstalling the PWA.
- Do not assume browser-local data is synchronized between devices.

Search and ranking

DocuLens uses lexical search with BM25-style frequency ranking, exact-phrase boosts and heading boosts. This favors term overlap and does not guarantee semantic equivalence.

- Search results should be checked in their surrounding document context.
- Exact phrase matching is documented as receiving a 3.0× multiplier.
- Heading text receives a ranking boost.
- Use more than one query formulation when a concept is described with different vocabulary.

Ranking signal	Interpretation	Limitation
Term frequency	Repeated terms may increase relevance	Repetition is not proof of importance
Exact phrase	Boosts precise phrase matches	Misses paraphrases
Heading boost	Elevates likely section context	Headings can be vague
Context review	User inspects passage and page	Requires human judgment

Inquiry and evidence retrieval

Inquiry starts with a question, retrieves relevant passages, and uses either a deterministic local path or optional remote synthesis. The evidence list should remain the basis for the final answer.

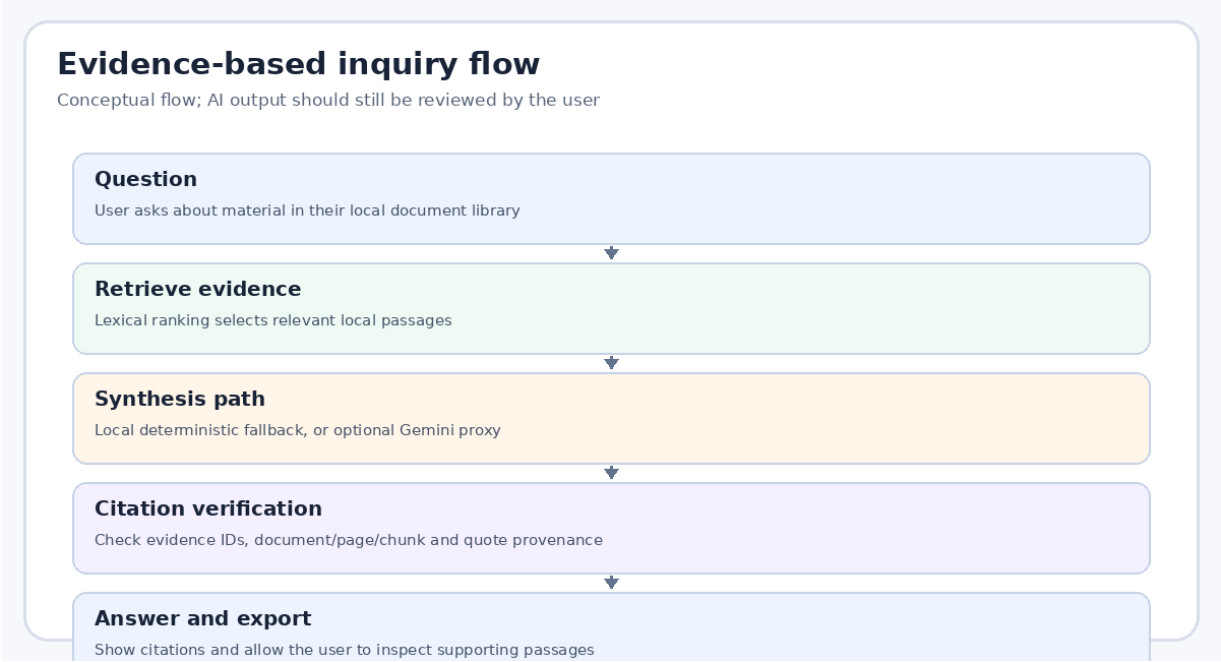


Figure 4. Evidence-based inquiry flow.

- The offline path can rank passages and construct a deterministic response.
- Remote synthesis sends a question and selected excerpts, not a complete PDF file.
- Treat the generated response as an aid, not a substitute for source review.
- Keep a record of the original question and cited passages for consequential work.

Citation verification model

Citation verification is intended to reject fabricated evidence IDs, nonexistent document references, invalid page references, missing chunks and quotes that do not match stored text.

- Evidence identifiers use bracketed tokens such as [ev-1].
- Text normalization tolerates whitespace variation for quote checks.
- A citation marked verified still requires a reader to assess whether it supports the claim.
- If source text is OCR-derived, errors in OCR can affect the quote match and interpretation.

Check	Why it matters
Evidence ID exists	Prevents references to non-retrieved evidence
Document exists	Prevents fabricated document identity
Chunk exists	Links citation to stored source text
Page is valid	Keeps navigation grounded
Quote matches	Checks verbatim provenance

Optional Gemini proxy

The server may call Google Gemini for synthesis when an API key is configured. The API key belongs in server environment configuration, never in the client bundle.

- GEMINI_API_KEY enables remote synthesis when configured.
- GEMINI_MODEL selects the model; verify the currently supported model name with the provider.
- Without a key or when remote AI is unavailable, local fallback is designed to remain available.
- Review provider terms and organizational policy before sending excerpts from sensitive documents.

Express server responsibilities

server.ts configures Express, compression, JSON body limits, security headers, rate limiting, proxy behavior and AI-related endpoints. The service is intended to be stateless with respect to full documents.

- JSON body limit: 2 MB.
- Inquiry endpoint rate limit: documented as 25 requests per minute per IP.
- Citation validation endpoint rate limit: documented as 60 requests per minute per IP.
- Request timeout and error behavior should be verified against the current server source.
- The server listens on process.env.PORT when supplied by the host.

Control	Observed design intent
Body cap	Reject oversized JSON requests
Rate limit	Reduce runaway requests and API consumption
Security headers	Reduce browser-side attack surface
Error sanitization	Avoid exposing internal stack traces and paths
Proxy trust	Interpret client IP when behind supported ingress

Security headers and request controls

The server sets content-type, referrer, permissions and content security policy headers. These are defense-in-depth controls, not a replacement for secure implementation and review.

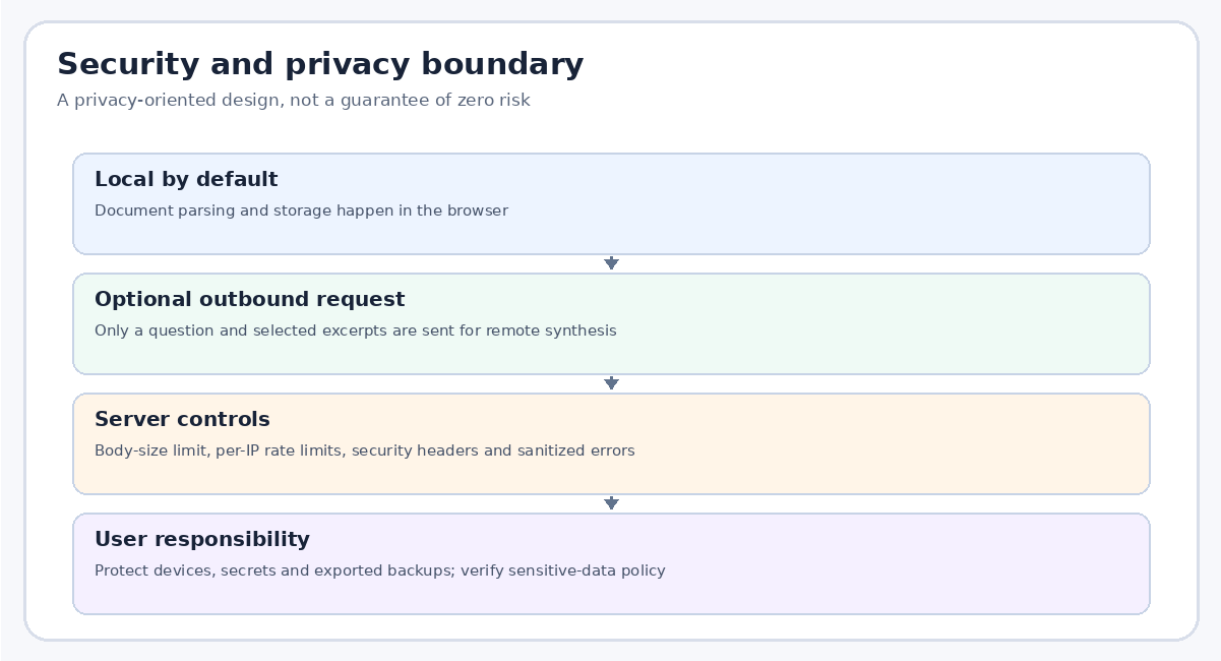


Figure 5. Security and privacy boundary.

- Do not loosen Content Security Policy without understanding each source directive.
- Avoid logging sensitive excerpts or secret values.
- Use HTTPS in production.
- Keep dependencies updated and review security advisories.
- Do not commit .env files or production credentials.

Privacy and outbound data

The local-first design keeps the full document in the browser during normal parsing and storage. Remote inquiry is a distinct pathway that can send the question and selected excerpts to the configured AI provider.

- Tell users clearly when remote synthesis is enabled.
- Do not represent the app as guaranteeing absolute privacy or zero data exposure.
- Use local fallback for documents that must not leave the device.
- Protect exported backups; they may contain extracted text and annotations.
- Check privacy policy and vendor terms before processing regulated information.

Tables and safe export

Table extraction attempts to reconstruct structured rows and columns from document content. The editor supports cell/row/column operations and CSV export.

- Review detected columns and headers before export.
- CSV output is designed to follow RFC 4180.
- CSV formula injection defenses are documented; still treat external spreadsheets as untrusted.
- Check encoding, delimiter behavior and totals after exporting important data.

Export	Use case	Review before sharing
CSV	Spreadsheet analysis	Headers, quoting, formula-like cells
Editable table	Correct extraction errors	Merged cells and inferred columns
Backup JSON	Restore workspace state	Contains local data and should be secured

Figures and OCR

Figure extraction supports raster figures, vector drawings and page scans as described by the project documentation. OCR can index text in diagrams to improve search.

- OCR is language- and image-quality-dependent.
- Inspect extracted figures at full size before using them as evidence.
- A figure's OCR text may not preserve spatial relationships, axes or legends.
- The bundled English model supports English recognition; other languages require additional setup.

Document comparison

Four comparison modes provide different lenses: concepts, definitions, topic overlap and revision differences. Each mode is a heuristic and should be validated against the original documents.

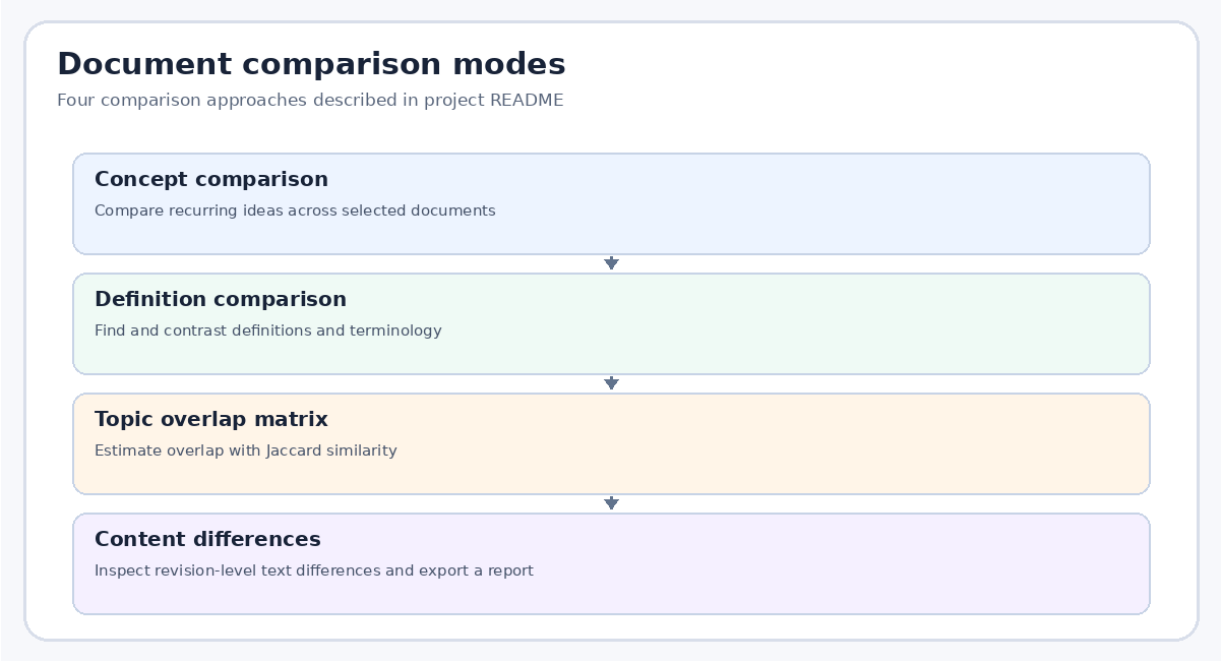


Figure 6. Comparison modes.

Mode	Question answered	Interpretation
Concept comparison	What ideas recur?	Qualitative grouping may need review
Definition comparison	How are terms defined?	Context affects meaning
Topic overlap matrix	How much vocabulary overlaps?	Jaccard similarity is lexical, not semantic
Content differences	What changed between revisions?	Formatting or extraction differences may appear

Backup and restore design

The project describes a schema-v1 JSON backup with atomic transaction restore and Merge or Replace All handling. Users should test a backup before relying on it as a recovery plan.

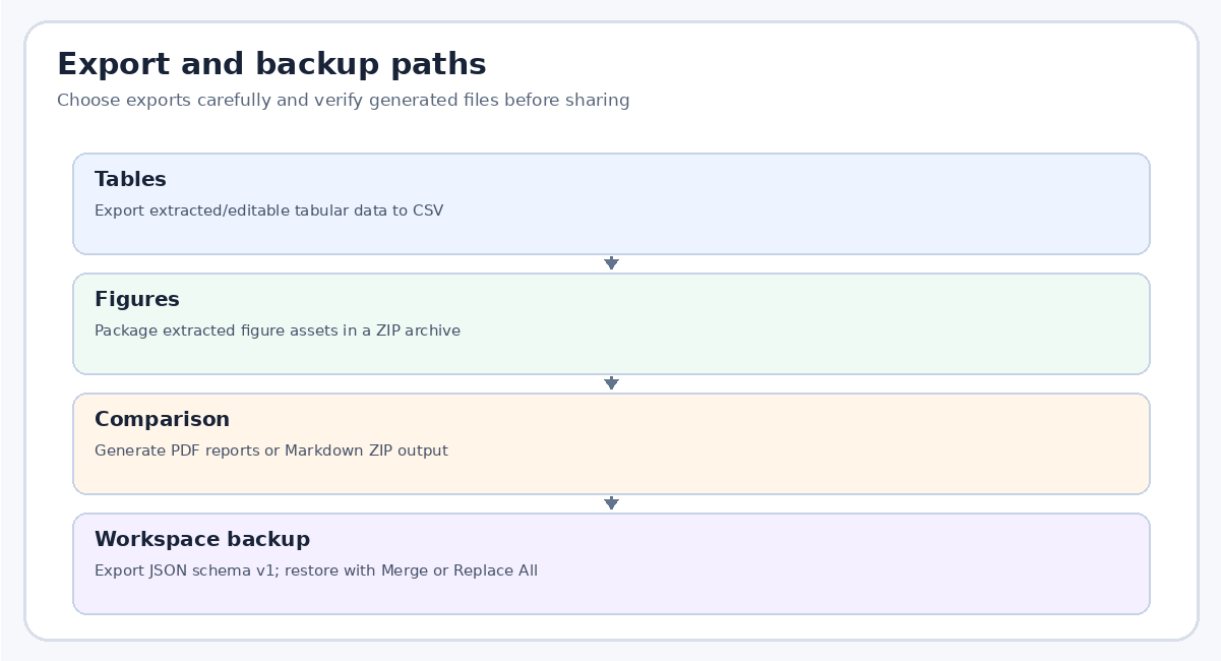


Figure 7. Export and backup paths.

- Export a backup after meaningful work sessions.
- Store backups in a controlled location with appropriate access restrictions.
- Use Merge to preserve existing workspace content when supported by the conflict rules.
- Use Replace All only when intentionally replacing the current local workspace.
- Validate record counts and open representative documents after restoration.

PWA and offline behavior

DocuLens is described as an installable Progressive Web App with versioned service-worker caching. Offline use depends on the app shell and required assets having been cached successfully.

- Install from a supported browser when the install prompt is available.
- A first visit may require a network connection to cache assets.
- OCR requires the bundled language model to be reachable and cached.
- A service-worker update may require reload or app restart.
- Test offline behavior after a successful online load.

Accessibility and interaction design

A document workbench should remain usable with keyboard navigation, readable contrast, descriptive controls and clear state changes. Confirm the current UI against these checks during release.

- Use meaningful labels for buttons and inputs.
- Ensure focus remains visible during dialogs and menus.
- Provide clear loading, empty, success and error states.
- Check text scaling and narrow viewport layouts.
- Do not rely on color alone to communicate citation verification or processing state.

Build pipeline and output artifacts

The build should produce a Vite output directory containing index.html and static assets. The Express production server expects the output directory at the path configured in server.ts.

- prebuild copies OCR assets before Vite build.
- build runs vite build.
- Verify dist/index.html exists after build.
- A successful server startup does not prove the frontend artifact exists.
- Check Render logs for missing-file errors such as ENOENT.

Check	Expected outcome
Dependency install	Lockfile-consistent dependency tree
OCR asset copy	eng.traineddata available to build
Vite build	dist/index.html and assets emitted
Production start	Express listens on host-provided PORT
HTTP smoke test	Root URL returns the app rather than an error

Render deployment checklist

For the supplied package, use the package manager and lockfile consistently. The repository includes both bun.lock and package-lock.json; avoid changing package managers casually.

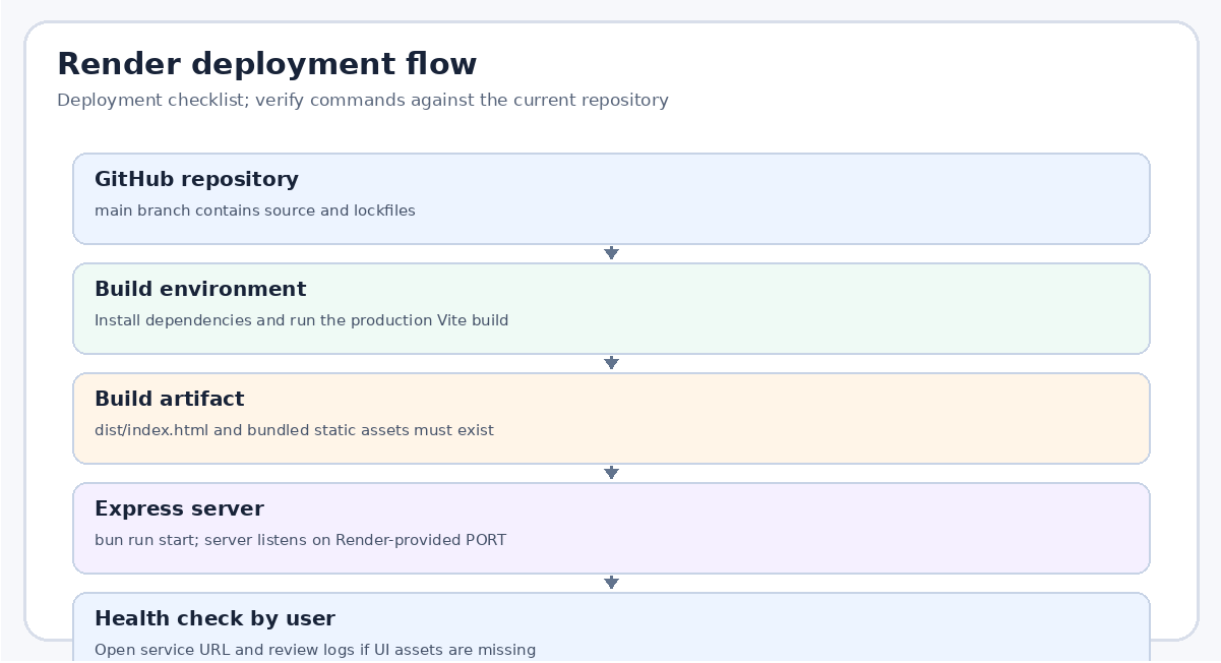


Figure 8. Deployment flow.

- Recommended Bun path: build command bun install && bun run build; start command bun run start.
- Confirm the Render service root is the repository root unless the app is in a subdirectory.
- Do not set a fixed PORT; Render supplies it.
- Add GEMINI_API_KEY only if remote AI is desired.
- After deploy, check logs and open the public URL; a live status alone is not a functional smoke test.

Environment variable reference

Environment variables configure optional AI and runtime URL/port behavior. Values differ between local development and hosted environments.

Variable	Purpose	Guidance
GEMINI_API_KEY	Optional remote AI credential	Set as secret in hosting dashboard
GEMINI_MODEL	Model selection	Use a provider-supported model name
APP_URL	Canonical deployed URL	Set only if application behavior relies on it
PORT	HTTP listener port	Let Render supply it; don't hardcode

Testing strategy

The repository includes Vitest tests for document management, table editing, backup/restore, citation navigation, OCR-related behavior, comparison and export behavior. Run the full suite before release.

- npm run lint checks TypeScript without emitting output.
- npm test runs the Vitest suite.
- npm run build validates the production frontend build.
- npm run verify chains lint, tests and build.
- Add regression tests for any bug fixed in parsing, citations, persistence or exports.

Layer	Example verification
Static checks	TypeScript compilation
Unit tests	Search, comparison and citation helper behavior
Storage tests	Backup/restore and document management
Build test	Production asset generation
Manual smoke test	Upload sample PDF, search, inquire, export and reload

Release and change management

Treat changes to the data model, citation format, backup schema, storage keys, server API or service-worker cache as compatibility-sensitive.

- Document migration behavior before changing IndexedDB stores.
- Keep backup schema compatibility explicit.
- Invalidate service-worker caches when required and test upgrades.
- Update README and docs when commands or limits change.
- Keep changelog entries focused on user-visible or operationally important changes.

Performance and reliability

Browser-based parsing shifts CPU and memory work to the user's device. Large PDFs, high-resolution scans and numerous extracted figures can be expensive.

- Use documented file and page limits as guardrails.
- Show progress during expensive parsing and OCR operations.
- Avoid repeatedly parsing unchanged content where caching is safe.
- Test on a mid-range device and a large-but-allowed sample document.
- When memory pressure appears, reduce batch size or inspect thumbnail/OCR processing.

Known limitations and risk register

The system relies on extraction quality, browser storage availability, provider availability and the correctness of its heuristics. The documentation should communicate these limits honestly.

Risk	Potential effect	Mitigation
Poor scan quality	Incorrect OCR text	Review source page and extracted text
Browser data cleared	Local workspace loss	Export backups regularly
Remote AI unavailable	No remote synthesis	Use deterministic fallback where supported
Lexical mismatch	Relevant passage not retrieved	Try alternate search wording
Deployment artifact missing	Blank/error page	Confirm dist/index.html and build logs